

Arrays and Conditions

Arrays

- Enumeration of values with the same data type

```
int[] numbers = new int[] { 1, 2, 3, 4, 5 }; //Length: 5, Indexes: 0 to 4  
string[] letters = new string[] { "a", "b", "c" }; //Length: 3, Indexes: 0 to 2  
double[] decimalNumbers = new double[10]; //Length: 10, Indexes: 0 to 9
```

- Access with a zero-based index

```
numbers[2]; //3  
letters[1]; // "b"  
letters[2] = "X"; //{ "a", "b", "X" }
```

Functions and Properties of Arrays

```
int[] numbers = new int[] { 1, 2, 3, 4, 5 };
```

Function	Description	Example
Length	Gets the length of the array	numbers.Length; //5
Contains(value)	Checks if the array contains the given value	numbers.Contains(5); // true
Max()	Returns the biggest value in the array	numbers.Max(); //5
Min()	Returns the smallest value in the array	numbers.Min(); //1
First()	Returns the first value in the array	numbers.First(); //1
Last()	Returns the last value in the array	numbers.Last(); //5
Sum()	Returns the sum of the array	numbers.Sum(); //15

Multidimensional Arrays

- An array can have multiple dimensions

```
//Declaration of a two-dimensional array
```

```
int[,] twoDimensions;
```

```
//Two-dimensional array with two lines and five columns
```

```
twoDimensions = new int[,] { { 1, 2, 3, 4, 5 }, { 6, 7, 8, 9, 10 } };
```

```
//Two-dimensional array with four rows and three columns
```

```
twoDimensions = new int[4, 3];
```

```
//Access the array with two indizes
```

```
int value = twoDimensions[1, 2];
```

```
//Assigning a value via two indizes
```

```
twoDimensions[1, 2] = 20;
```

Functions and Properties of multidimensional Arrays

```
int[, ,] array3D;  
array3D = new int[2, 3, 2] { { { 1, 2 }, { 3, 4 }, { 5, 6 } }, { { 7, 8 }, { 9, 10 }, { 11, 12 } } };
```

Function	Description	Example
Length	Gets the length of the array	array3D.Length; // 12
Rank	Gets the number of dimensions	array3D.Rank; // 3
GetLength(int)	Returns the length of a given dimension	array3D.GetLength(0); // 2 array3D.GetLength(1); // 3 array3D.GetLength(2); // 2

Boolean Logic

- Each operation returns either **true** or **false**

```
a == b; //Is a equal to b?
```

```
a != b; //Is a not equal to b?
```

```
a < b; //Is a less than b?
```

```
a > b; //Is a greater than b?
```

```
a <= b; //Is a less than or equal to b?
```

```
a >= b; //Is a greater than or equal to b?
```

- Operations can be combined

```
x && y; //are both x and y true?
```

```
x || y; //is either x or y true?
```

```
x ^ y; //are x and y different (true/false or false/true)?
```

```
!x; //invert x, if x is true it becomes false and vice-versa
```

Conditions

```
if (Condition A)
{
    //if Condition A is true, run the code inside here
}
else if (Condition B)
{
    //if Condition A is not true but Condition B is, run the code inside here
}
else
{
    //if Condition A and B are false
}
```